

11. Validate Variable using Lex and YACC

variable.l

```
%{
#include "y.tab.h"
%}

%%

[a-zA-Z][a-zA-Z0-9]* { return IDENTIFIER; }
\n { return NL; }
. { return yytext[0]; }

%%

int yywrap(void) {
    return 1;
}
```

variable.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
%}

%token IDENTIFIER NL

%%

stmt:
    IDENTIFIER NL { printf("Valid Variable\n"); exit(0); }
;

%%

void yyerror(const char *s) {
    printf("Invalid Variable\n");
}
```

```
    exit(0);
}

int main(void) {
    printf("Enter a variable name: ");
    yyparse();
    return 0;
}
```

Output

Enter a variable name: x
Valid Variable

12. Calculator Using Lex and YACC

calc.l

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"

extern int yylval;
%}

%%

[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[ \t] { /* ignore spaces */ }
\n { return NL; }
. { return yytext[0]; }

%%

int yywrap(void) {
    return 1;
}
```

calc.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
void yyerror(const char *s);
}%

%token NUMBER NL

%left '+' '-'
%left '*' '/'

%%

stmt:
    expr NL { printf("Result = %d\n", $1); exit(0); }
    ;

expr:
    expr '+' expr    { $$ = $1 + $3; }
  | expr '-' expr    { $$ = $1 - $3; }
  | expr '*' expr    { $$ = $1 * $3; }
  | expr '/' expr    {
        if ($3 == 0) {
            printf("Division by zero error\n");
            exit(1);
        }
        $$ = $1 / $3;
    }
  | '(' expr ')'     { $$ = $2; }
  | NUMBER           { $$ = $1; }
    ;

%%

void yyerror(const char *s) {
    printf("Invalid Expression\n");
    exit(1);
}

int main(void) {
```

```
    printf("Enter expression: ");
    yyparse();
    return 0;
}
```

Output

```
Enter expression: 4+5-2
Result = 7
```